Conclusion DLISP Lab 2

Carlos Celaya, Marcel Hofmann

November 21, 2025

1 Tables of Experiments

Config Param	Standard	Exp. 1	Exp. 2	Exp. 3	Exp. 4	Exp. 5
Num Classes	4	4	4	4	4	4
Learning Rate	0.1	0.1	0.1	0.1	0.1	0.1
Train Batch Size	80	80	80	80	80	80
Test Batch Size	100	100	100	100	100	100
Epochs Pretext	10	45	45	45	45	45
Milestones Pretext	[7, 9]	[37, 45]	[37, 45]	[37, 45]	[37, 45]	[37, 45]
Epochs Classification	10	45	45	45	45	45
Milestones Classification	[7, 9]	[37, 45]	[37, 45]	[37, 45]	[37, 45]	[37, 45]
MaxImageNum	15*480	15*480	15*480	15*480	15*480	50*480
Seed	271828	271828	271828	271828	271828	271828
Momentum	0.9	0.9	0.9	0.9	0.9	0.9
Weight Decay	1e-4	1e-4	1e-3	1e-2	1e-2	1e-2
Gamma	0.1	0.1	0.1	0.1	0.1	0.1
Model Architecture	[2,1,2,1]	[2,1,2,1]	[2,1,2,1]	[2,1,2,1]	[1,1,1,1]	[3,2,3,2]
Pretext best val acc	-	70.72	73.5	20.01	73.69	78.56
Semisupervised best val acc	-	73.38	72.94	19.94	73.83	69.28
Scratch best val acc	-	66.0	67.94	71.89	67.16	Colab died

Table 1: Carlos' Experiment Table

Config Param	Exp. 1	Exp. 2	Exp. 3	Exp. 4
Use Inception	False	True	False	True
Num Classes	4	4	4	4
Learning Rate	0.1	0.1	0.1	0.2
Train Batch Size	256	256	256	256
Test Batch Size	256	256	256	256
Epochs Pretext	40	25	30	35
Milestones Pretext	[20, 30]	No	No	[30]
Epochs Classification	40	45	45	45
Milestones Classification	[20, 30]	[35, 40]	[35, 40]	[35, 40]
MaxImageNum	30*480	50*480	50*480	50*480
Seed	271828	271828	271828	271828
Model Architecture	[2,1,2,1]	[2,1,2,1]	[2,1,2,1]	[2,1,2,1]
Channel Widths	[64, 128, 256, 512]	[16, 32, 64, 128]	[64, 128, 256, 512]	[8, 16, 32, 64]
Num Params Main Classifier	6.168.143	1.346.418	6.168.143	354.738

Table 2: Marcel's Experiment Table

2 Training the Pretext Task

A: Include the graphical representations for the evolution of the training and validation losses and accuracies obtained when a ResNet-14 CNN is trained for predicting rotations using the default dataset and suited hyperparameters (see the Colab notebook).

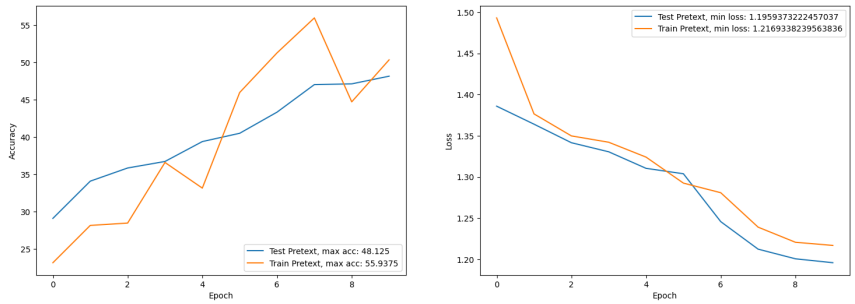


Figure 1: Loss and Accuracy for Rotation Prediction with standard config

The initial training and validation curves show that, with the default hyperparameters, the behaviour of the network is not sufficiently informative to determine whether a deeper model is required or whether overfitting is already present (Figure 1). The losses for both splits evolve smoothly, and the accuracy gap between training and validation remains moderate, which suggests that the model capacity is approximately adequate for the task. To obtain a clearer diagnostic, the number of training epochs was increased:

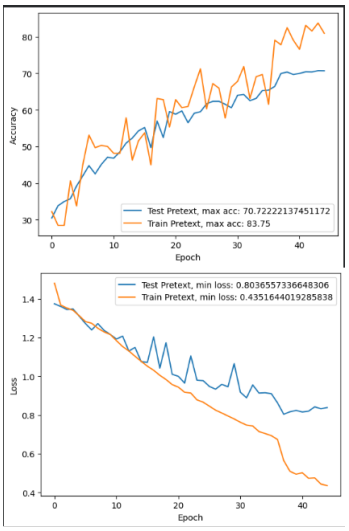
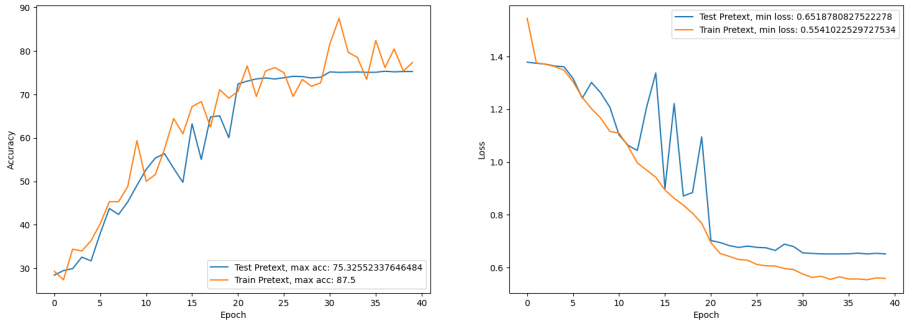


Figure 2: Loss and Accuracy for Rotation Prediction with increased epochs

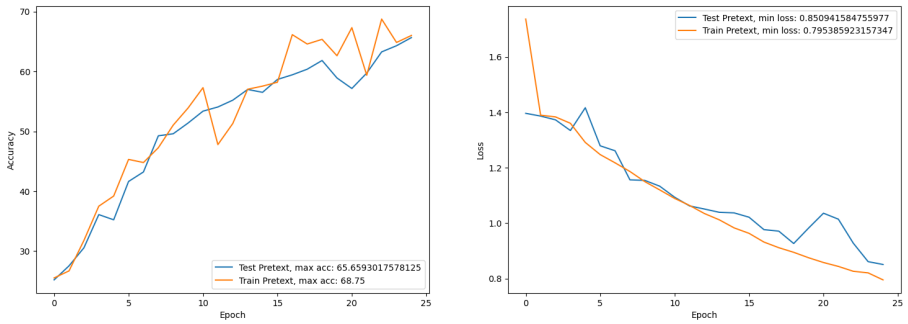
With this extended training schedule, a clear overfitting pattern emerged: the training accuracy continued to improve steadily, whereas the validation accuracy stagnated and eventually diverged, reaching a gap of 13.03%. This indicates that the model was memorising the training data rather than learning generalisable features for rotation prediction.

Given this behaviour, the first adjustment was to modify the weight decay parameter in an attempt to reduce overfitting. By increasing its value, the model's effective capacity is constrained, encouraging smaller weights and improving generalisation. As shown in the results, this modification initially led to a noticeable improvement in validation accuracy, increasing from 70.72% to 73.50%. However, increasing weight decay too aggressively had the opposite effect: with a value of $1e-2$, the model's performance collapsed to 20%, indicating severe underfitting. After testing multiple configurations, the most suitable compromise was found at $1e-3$, providing the best validation accuracy within this set of experiments.

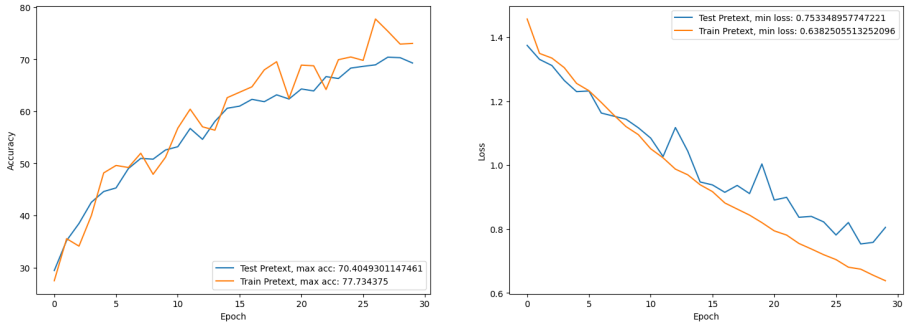
Apart from regularization using weight decay, other experiments were performed based on Table 2. The first experiment from this table results from the promising results of the standard experiment (standard settings). More data was used, the amount of epochs and the according milestones were set higher to match the epoch ratio from the beginning. Since the hardware allows it, for all experiments, the batch size was set higher. For having more variance in the architecture, an `InceptionBlock` with residual connection was implemented as alternative for the `BasicBlock`. It is used in experiment 2 with the full amount of data (leading also to have more classes). An experiment with the full amount of data was also performed with the basic ResNet block (experiment 3). The last experiments also includes further data augmentation techniques.



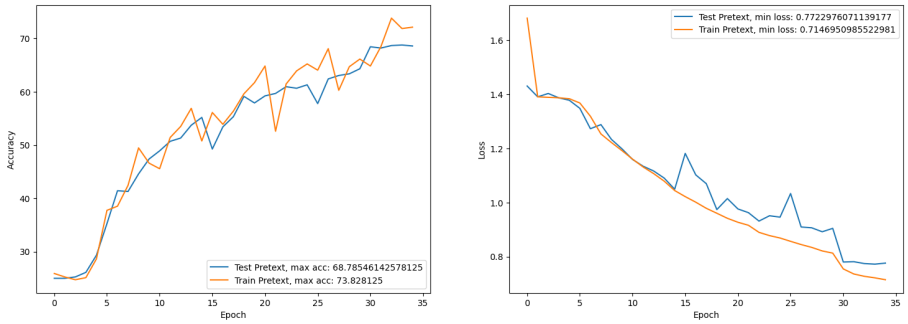
(a) Exp. 1 (increased epochs with base model)



(b) Exp. 2 (inception model with full data, less epochs)



(c) Exp. 3 (base model with full data)



(d) Exp.4 (inception model with advanced data augmentation)

Figure 3: Loss and Accuracy for Rotation Predictions based on experiments in Table 2

For the pretext task, generally, no real overfitting can be detected. Since the weight decay milestones have only tiny almost not noticeable effects (cf. exp. 1, from epoch 20 no progress because of milestone decreasing lr), they were either left out entirely or put for a small amount of epochs at the end. In general, using bigger datasets that introduce more classes, lead to higher difficulty resulting in poorer performance than with smaller datasets of fewer classes. Still, the last inception model being pretty small in comparison to the other ResNets, leads to almost same accuracy results.

B: Discuss the results and the potential benefits of using different training hyperparameters, a deeper/different architecture or a larger dataset. Optional: Extra points will be given if the discussion is supported with experiments.

Even after applying regularisation, the model continued to show overfitting tendencies (Table 1). This suggested that simply increasing depth would not be effective, since a deeper network would likely amplify the overfitting instead of mitigating it. For this reason, a complementary approach was tested: reducing the number of layers.

The architecture was simplified from [2,1,2,1] basic blocks for all the neural networks to [1,1,1,1], aiming to reduce the model capacity and improve generalisation. This modification, however, produced almost no meaningful improvement, with the validation accuracy increasing by only 0.13%. This minimal difference indicates that the performance bottleneck is not necessarily the model complexity.

The next hypothesis considered was that the dataset might be too limited for the task. A small dataset can cause unstable optimisation and restrict the ability of the model to learn rotation-related visual patterns, regardless of regularisation strategies. To test this idea, the amount of training data was increased by expanding the number of image classes from 15 to 50, while simultaneously increasing the depth of the architecture to [3,2,3,2] basic blocks.

This modification produced the best pretext accuracy so far (78.56%), supporting the hypothesis that the network benefits significantly from a larger dataset. With more samples, the model can rely less on memorisation and instead develop more robust features, which improves performance even with increased depth.

The inception network can provide similar results as basic ResNets that contain far more trainable parameters. This is probably because of the high amount of convolutional operations of different sizes. It makes it effective but less efficient. Convolutions are very computationally expensive (approx. $O(n) = n^4$). This makes the network very slow (often even slower than the basic ResNets). Because of that, the amount of channels has been decreased (reducing slowliness and trainable params).

3 Training the Classification Task

A: Include the graphical representations for the evolution of the training and validation losses and accuracies obtained when a ResNet-14 CNN is trained for classification using the default dataset and hyperparameters specified in the Colab notebook and starting from the weights learned through the pretext task.

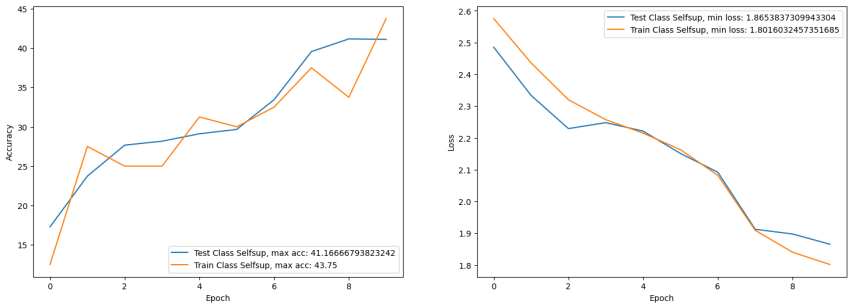


Figure 4: Loss and Accuracy for Classification using Pretext-trained with standard configuration and using random weight initialization with standard configuration

Using the default dataset and hyperparameters, the classification network was first trained by initializing its weights with those learned during the pretext task. As in the pretext stage, the initial number of epochs was insufficient to observe clear behaviour in terms of convergence or generalisation, which motivated increasing training to 45 epochs (Figure 4).

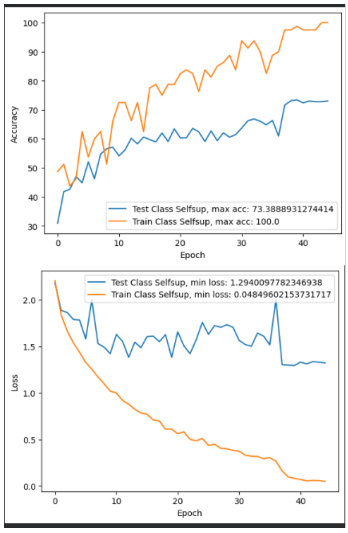
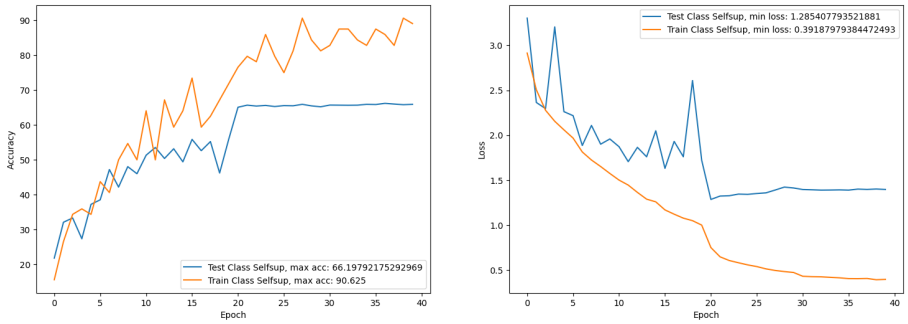


Figure 5: Loss and Accuracy for Classification using Pretext weights

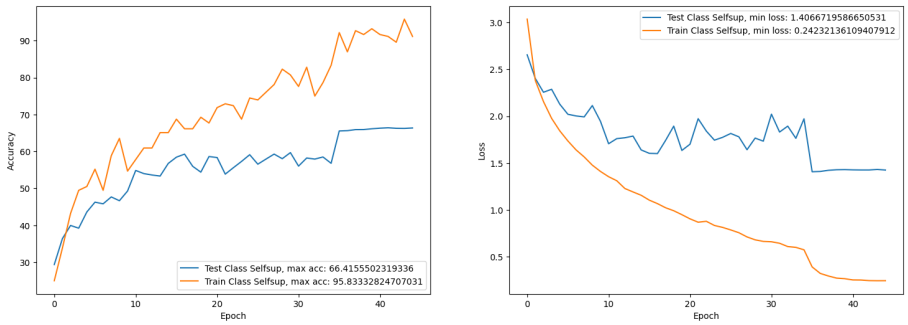
Once extended training was applied, the limitations of the model became more visible. Since the context network (RotNet) was already overfitted during the pretext task, the transferred weights inherited part of that bias: the training accuracy increases rapidly, while the validation accuracy saturates earlier and eventually diverges. In practice, this manifests as feature representations that are highly tuned to the pretext rotations but not perfectly suited to the downstream classification task.

To address this issue, the same regularisation strategies tested in the pretext stage were applied: mainly adjusting the weight decay parameter. Surprisingly, in this classification phase the increase in regularisation produced a decrease in accuracy. This behaviour can be explained by the fact that the pretrained features already encode useful rotation-related patterns; applying strong penalties on the weights suppresses these learned representations, effectively "erasing" information acquired during self-supervised training. Instead of reducing overfitting, the excessive regularisation.

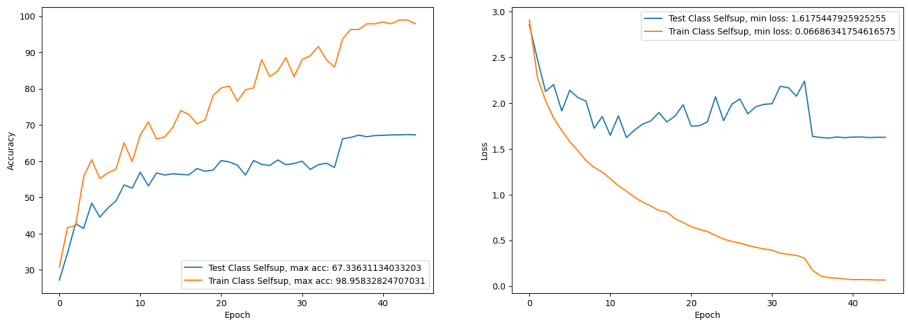
The plots of the experiments from Table 2 look as follows:



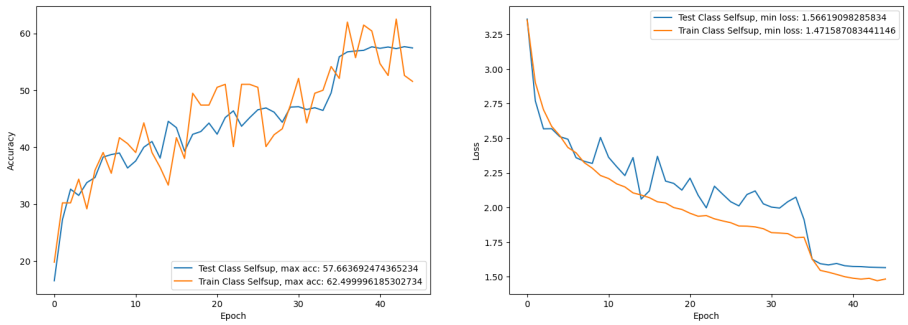
(a) Exp. 1 (increased epochs with base model)



(b) Exp. 2 (inception model with full data, less epochs)



(c) Exp. 3 (base model with full data)



(d) Exp.4 (inception model with advanced data augmentation)

Figure 6: Loss and Accuracy for Class Predictions using weights from pretext training, based on experiments in Table 2

Training on the full dataset leads to strong overfitting. This is because with more parts of the whole data, new classes are introduced. Generally, there are too few images to train on a too high amount of classes. It leads to bad generalization. For the first experiment (not with the entire dataset), this effect is not as strong as with more data. It is noteworthy that the smaller inception model tends more to overfitting (cf. experiment 3). It scratches 100% accuracy for training images. When not having enough data, data augmentation needs to be applied. Decreased overfitting can be seen in the last experiment (experiment 4). Methods like flipping, rotation and color jittering are used. This leads to far more training time because each image needs to be preprocessed using the transform steps defined in the script. Because of that, a smaller inception model was used to mitigate this effect a bit (more channels for inception would cause far more training time). Smaller models also tend to generalize better (fighting overfitting) since they have not that much trainable parameter space to learn the training data "by heart". As a result of using augmented data and a smaller model, in the plot of experiment 4, almost no overfitting can be examined.

B: Include a comparison of the graphical representations yielded in the previous stage with those obtained when a ResNet-14 CNN is trained for classification by starting from random weights (from scratch) and using the default dataset and the same hyperparameters as in the previous stage.

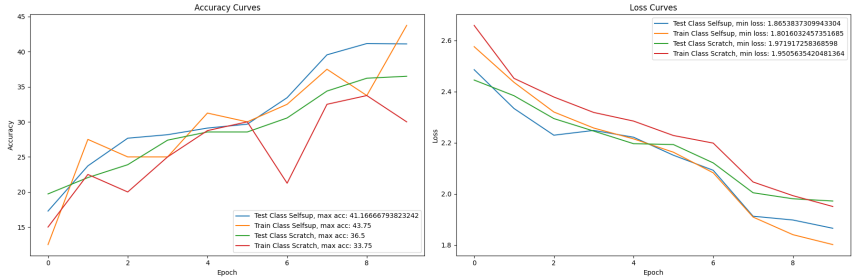
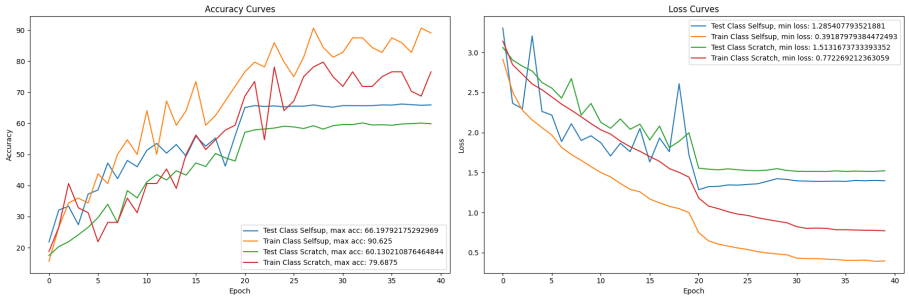


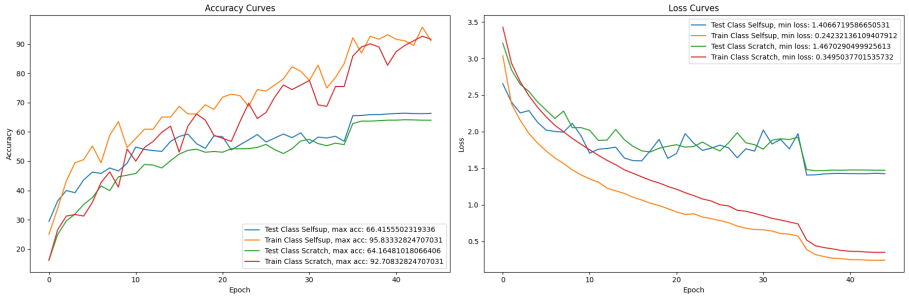
Figure 7: Loss and Accuracy curves for Classification using Pretext-trained weights and randomly initialized weights (scratch)

The plot of the random weigths from scratch can also be seen in the Figure 7.When the same classification task was repeated starting from random weights, the training dynamics changed substantially. In contrast to the pretrained model, this network has to learn all low-level and mid-level features from zero, which makes it significantly more prone to noise and overfitting. Interestingly, in this case higher levels of weight decay improved accuracy. This occurs because, without pretrained features, the model is more flexible but also more unstable. Regularisation helps constrain the growth of irrelevant filters, providing smoother optimisation and guiding the network towards more generalizable solutions. However, despite the improvements obtained through increased regularisation, the scratch model consistently performed worse than the pretrained one. Even its best result did not surpass the worst-performing configuration of the self-supervised model. This confirms that the rotation-based pretraining gives access to more structured and transferable visual representations, reducing the burden placed on the classification stage.

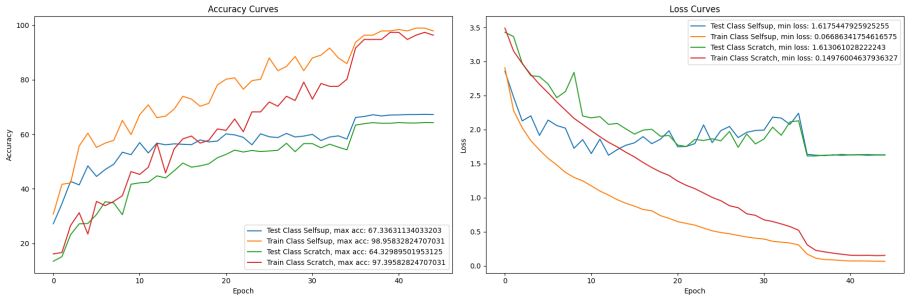
A similar trend can be seen in the experiments in table 2. The curves can be more easily differentiated for the overfit experiments. Overfitting for the pretext weights is the same as overfitting for the randomly initialized weights:



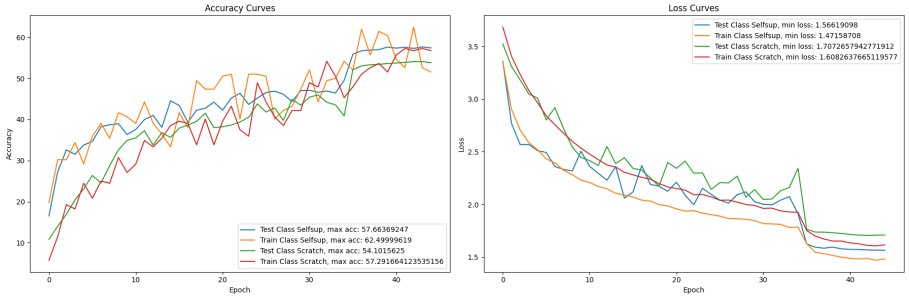
(a) Exp. 1 (increased epochs with base model)



(b) Exp. 2 (inception model with full data, less epochs)



(c) Exp. 3 (base model with full data)



(d) Exp.4 (inception model with advanced data augmentation)

Figure 8: Comparison of Loss and Accuracy for Class Predictions with random weight initialization and pretext weights based on experiments in Table 2

C: Discuss the results and the benefits of starting from self-supervised learned weights.

The results clearly show that initializing the classification network with weights learned through the self-supervised rotation task provides a substantial advantage over training the model from scratch. Across all experiments, the pretrained model consistently achieves higher validation accuracy, even in those configurations where excessive regularization reduces its performance. This improvement stems from the quality of the representations acquired during the pretext task: by learning to predict rotations, the network develops filters that capture orientations, contours, and other low-level geometric patterns that are immediately useful for downstream classification. As a consequence, optimization during the supervised stage becomes more stable and efficient, since the model does not need to construct a complete hierarchy of visual features from random initialization. The fact that the worst-performing configuration of the pretrained model still surpasses the best-performing configuration of the scratch model further confirms that self-supervision acts as an effective form of initialization, preventing the network from falling into poor minima and improving its ability to generalize.

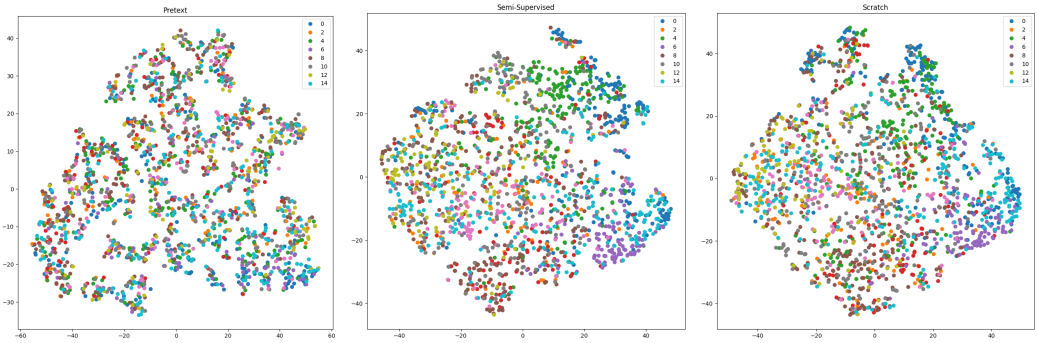
D: Discuss the potential benefits of using different training hyperparameters, a deeper/different architecture, or larger datasets. Optional: Extra points will be given if the discussion is supported with experiments.

The experiments conducted reveal that modifications to hyperparameters, architectural depth, and dataset size affect the pretrained and scratch models in fundamentally different ways, highlighting how strongly training dynamics depend on the initial state of the network (Table 1). For the pretrained model, reducing the depth of the architecture produced a very small improvement, increasing the best validation accuracy from 72.94% to 73.83% (+0.89%), which suggests that a more compact network can make more efficient use of the rotation-based features learned during the pretext task and may reduce overfitting in the supervised stage. In contrast, increasing the architectural depth led to a substantial drop in performance for the pretrained model, falling from 73.69% to 69.28% (-4.41%), likely because the newly added layers were not optimized during pretraining and disrupted the structure of the learned representations. The scratch model exhibited the opposite tendency: while the shallower configuration reached 67.16%, adding depth improved performance to 71.89% (+4.73%), reflecting the model's need to develop rich hierarchical features from randomness and its reliance on greater representational capacity. The effect of weight decay also differed sharply between the two models. In the scratch network, increasing weight decay from 1e-4 to 1e-3 raised accuracy from 66.0% to 67.94% (+1.94%), and further increases continued to stabilize learning, whereas applying the same higher regularization to the pretrained model caused severe deterioration, collapsing accuracy from 73.5% to 19.94% (-53.56%) when the decay was excessively large, as the penalty erased valuable information acquired during the pretext task. Finally, enlarging the dataset had a clearly positive influence, particularly on the pretext stage, where accuracy increased from 73.69% to 78.56% (+4.87%), which subsequently enabled stronger and more generalizable rotation-based representations.

That different hyperparameters can have huge effects on training, can also be seen in the experiments of Table 2.

4 Model Visualizations

A: Include the t-SNE representations of the three learned models: pretext-trained, fine-tuned for classification from the pretext-trained, and trained from scratch for classification.

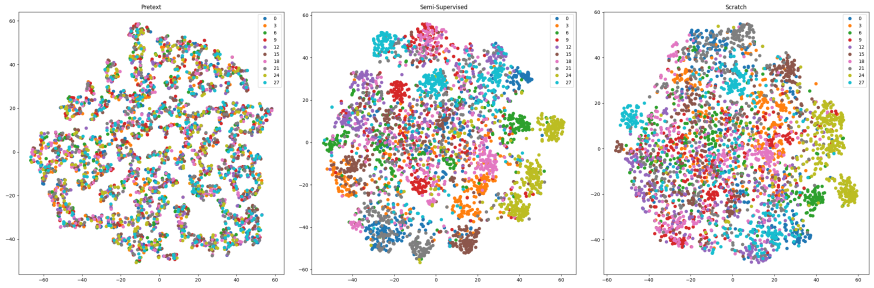


(a) Pretext rotation classifier (b) Classifier using pretext weights (c) Classifier using random init weights

Figure 9: T-SNE representation of model predictions using standard config

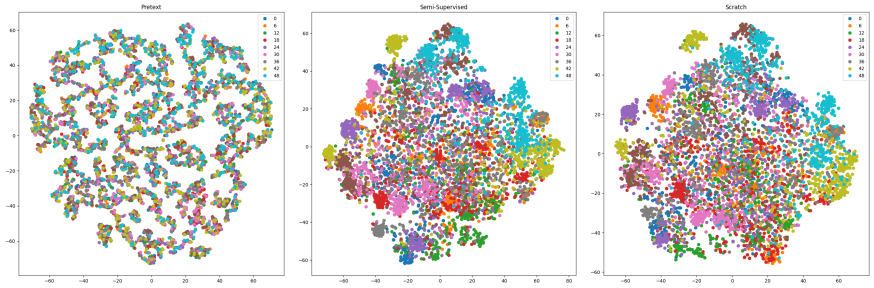
The t-SNE visualizations reveal notable differences in how the three models structure the feature space (Figure 9). The representation obtained from the pretext-trained model shows little to no visible class separation, with points distributed in a diffuse and disorganized manner, indicating that the rotation-based task alone does not produce clusters aligned with the downstream semantic categories. In contrast, the fine-tuned model trained for classification starting from the pretext weights exhibits a much clearer formation of class-dependent regions, where several groups become noticeably compact and internally coherent, even though some misclustered samples persist, particularly near the boundaries between classes. Finally, the model trained from scratch also produces clusters but they are less coherent and more fragmented than those of the fine-tuned model; although some degree of separation is present, the distances within clusters tend to be larger and the overlap between classes is noticeably higher.

The plots of the experiments of table 2 show the mentioned clusters well (cf. next page). Generally, many clusters are clearly visible. Since in the first amount not the whole dataset was used, the plot looks not that full as the others. Apparently, the classifiers using the pretext weights succeed better at separating samples cluster-wise than the classifiers using random weight init. Looking at the clusters, the pretext weight classifier of experiment (Fig. 12) seems to be the best one. The cluster separation is the cleanest. Experiment 4 is the one that looks worst but one need to keep in mind that this experiment contained data augmentation. The training of these classifiers should be continued as loss and accuracies seem not to be fully converged yet (probably leeding to clearer clusters) (cf. Fig. 8). Keeping in mind that the models are far smaller than the other ones used in the experiments, their results are very convincing.



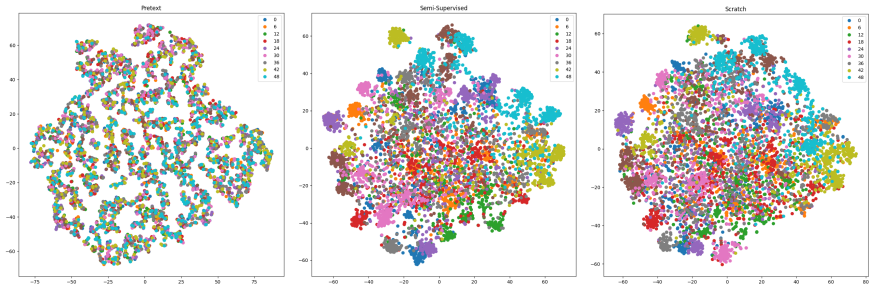
(a) Pretext rotation classifier (b) Classifier (pretext weights) (c) Classifier (random init)

Figure 10: T-SNE representation of models of experiment 1



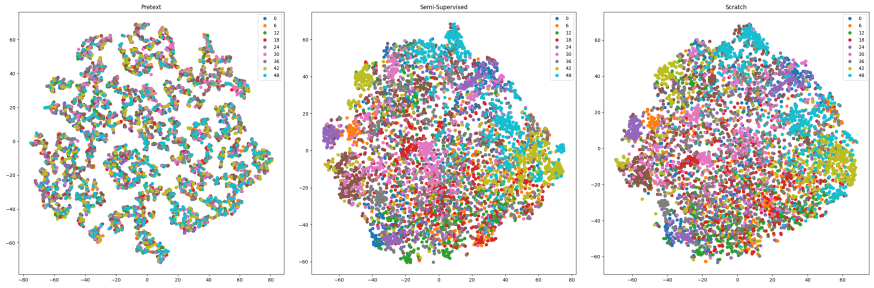
(a) Pretext rotation classifier (b) Classifier (pretext weights) (c) Classifier (random init)

Figure 11: T-SNE representation of models of experiment 2



(a) Pretext rotation classifier (b) Classifier (pretext weights) (c) Classifier (random init)

Figure 12: T-SNE representation of models of experiment 3



(a) Pretext rotation classifier (b) Classifier (pretext weights) (c) Classifier (random init)

Figure 13: T-SNE representation of models of experiment 4

B: Discuss the results and the benefits of starting from self-supervised learned weights and comment on their capacity to linearly separate the classes and on their space coverage.

The t-SNE results demonstrate that starting from self-supervised learned weights yields feature spaces that are more amenable to linear separation and exhibit better semantic organization than those obtained when the model is trained from scratch. The representation of the fine-tuned network covers the space more efficiently, creating compact clusters whose boundaries are sharper and whose internal distances are smaller, which indicates that the underlying features encode more discriminative information aligned with the classification task. By contrast, the scratch model spreads the data more irregularly, forming clusters that are less concentrated and more prone to overlap, reflecting the slower emergence of meaningful structure when beginning from random initialization. The pretext model, which has not yet been exposed to class labels, occupies the feature space in an almost uniform and undifferentiated manner, revealing that rotation prediction alone does not impose semantic grouping but nonetheless provides a geometric prior that becomes useful once supervised fine-tuning begins.